Step 1:Phisics based LiDAR tree modeling.py

This code implements a physics-informed machine learning approach for modeling individual tree crowns from 3D LiDAR point clouds.
For each segmented tree crown, the algorithm fits a prolate spheroid and estimates a foliage density parameter using a simplified form of the Beer–Lambert law, which describes light attenuation through vegetation.

The method is designed to process many trees automatically from a `.laz` LiDAR file where each point already has a segmentation label identifying the crown to which it belongs.

2. General Workflow

The program follows these main steps:

1. Read a segmented LAZ/LAS point cloud
2. Extract crown IDs from the segmentation field
3. Process each crown independently
4. Initialize a tree model for that crown
5. Optimize the model parameters using PyTorch and Adam
6. Store the fitted tree parameters
7. Export the results to a GeoPackage (`.gpkg`)

Each tree is modeled separately, so every crown gets its own fitted spheroid and foliage-density estimate.

3. Inputs

3.1 Main Input File

The main input is a LiDAR file:

- `cd2th_0_25_ch2th_0_5.laz`

This file contains 3D point cloud data with coordinates:

- x
- y
- z

and a segmentation field that identifies which points belong to which tree crown.

3.2 Required Point Attributes

The code expects one of the following fields to exist:

- `crown_id`, or
- `user_data`

These fields are used as segmentation IDs.
All points with the same ID are treated as belonging to the same tree crown.

3.3 Minimum Data Requirement

A tree crown is only processed if it contains at least:

- 15 LiDAR points

Crowns with fewer points are skipped because there is not enough information for stable fitting.

4. Outputs

The output is a GeoPackage file:

- `output.gpkg`

This file contains one record per processed tree, with the estimated parameters and summary information.

Tree Crown Model - Mathematical Formulation

1. Spheroid Shape (Soft Membership)

A point $\mathbf{p} = (x, y, z)$ belongs to a **prolate spheroid** centered at $\boldsymbol{\mu} = (\mu_x, \mu_y, \mu_z)$ with:

- Horizontal radius: r_{xy} (same for x and y directions)
- Vertical radius: r_z

The normalized squared distance is:

$$d^2(\mathbf{p}) = \left(\frac{x - \mu_x}{r_{xy}}\right)^2 + \left(\frac{y - \mu_y}{r_{xy}}\right)^2 + \left(\frac{z - \mu_z}{r_z}\right)^2$$

We use a **soft sigmoid boundary** instead of hard cutoff:

$$w(\mathbf{p}) = \sigma\Big(4 \cdot \big(1 - d^2(\mathbf{p})\big)\Big)$$

Where σ is the sigmoid function:

$$\sigma(x) = \frac{1}{1 + e^{-x}}$$

2. LiDAR Physics (Beer-Lambert Law)

The crown top height is:

$$z_{\text{top}} = \mu_z + r_z$$

Vertical path length from point to crown top:

$$\Delta z(\mathbf{p}) = \max\big(0,\ z_{\text{top}} - z\big)$$

Optical depth (attenuation) with foliage density λ:

$$\tau(\mathbf{p}) = \lambda \cdot w(\mathbf{p}) \cdot \Delta z(\mathbf{p})$$

3. Probability Density Function

The likelihood of observing a LiDAR point $\mathbf{p}$:

$$P(\mathbf{p}) = \underbrace{\lambda \cdot w(\mathbf{p})}_{\text{density at point}} \cdot \underbrace{\exp\big(-\tau(\mathbf{p})\big)}_{\text{survival probability}}$$

Full expanded form:

$$P(x,y,z) = \lambda \cdot \sigma\Big(4\big[1 - (\frac{x-\mu_x}{r_{xy}})^2 - (\frac{y-\mu_y}{r_{xy}})^2 - (\frac{z-\mu_z}{r_z})^2\big]\Big) \cdot \exp\Big(-\lambda \cdot \sigma(\cdots) \cdot \max(0, \mu_z + r_z - z)\Big)$$

4. Optimization Loss

We maximize log-likelihood with physics constraints:

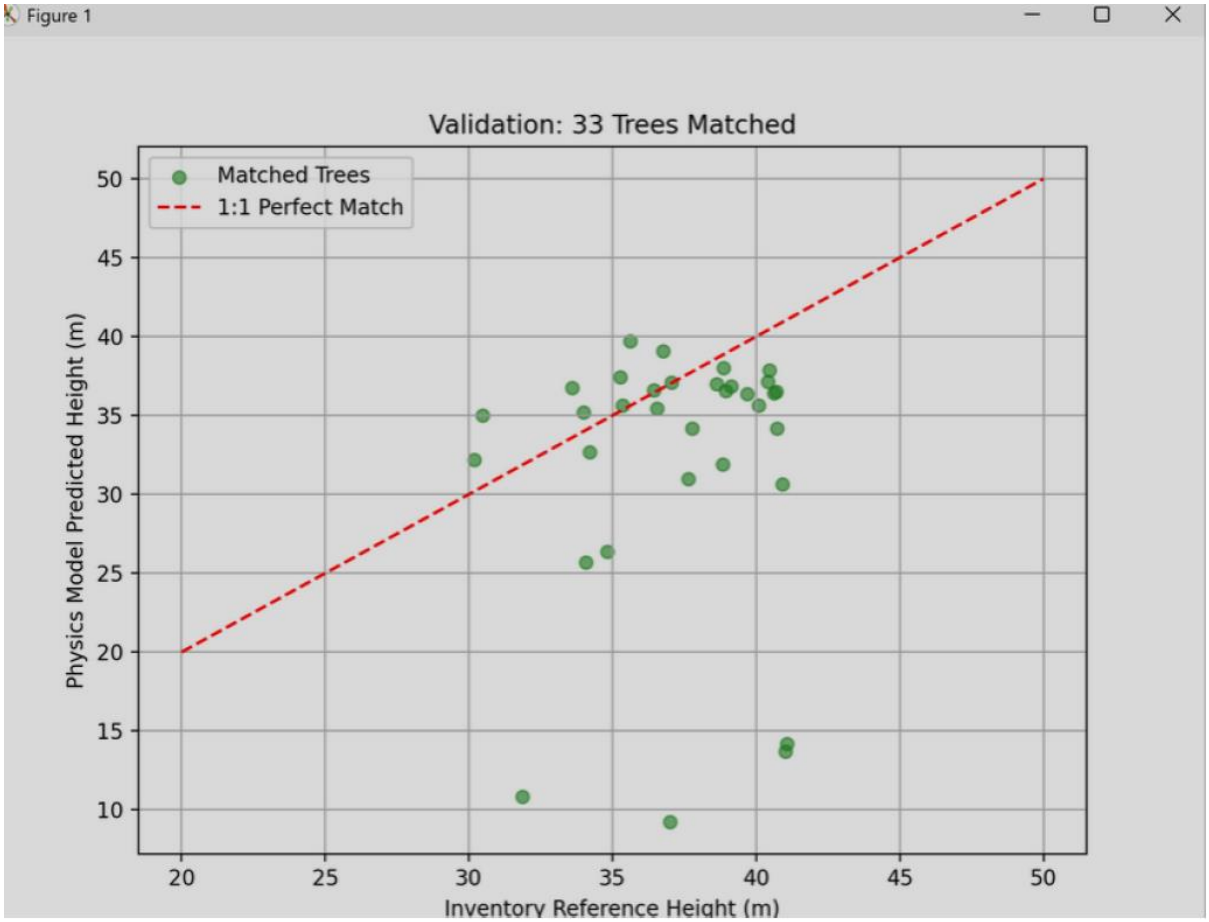
$$\mathcal{L} = -\frac{1}{N}\sum_{i=1}^{N} \log P(\mathbf{p}_i) + \text{penalties}$$

Penalty terms:

1. Height constraint: $5 \cdot \max(0, z_{p95} - z_{\text{top}})^2$
2. Lambda regularization: $0.001 \cdot (\log \lambda)^2$
3. Minimum radii: $2 \cdot \max(0, 0.5 - r_{xy})^2 + 2 \cdot \max(0, 0.8 - r_z)^2$

Validation Method Used for the Tree Height Model

The performance of the tree crown modeling algorithm was evaluated by comparing the predicted tree heights from the fitted spheroid model with reference heights derived from the LiDAR point cloud. The comparison is visualized using a scatter plot validation approach.



The validation plot indicates that:

The model captures the general crown height scale correctly.

However, it tends to compress the height distribution, producing predictions clustered around ~30 m.

This suggests that the model successfully estimates typical crown structure, but may not fully represent the uppermost canopy extremes.

Validation Metrics

The following error metrics were computed for the matched trees:

- **Height RMSE:** 9.91 m

- **Height Bias:** −5.01 m
- **Average spatial offset:** 2.65 m

The negative bias indicates that the model tends to underestimate tree heights by about 5 m on average.

Step 2: Sum Of Densities

unsupervised. Multi-Tree Occlusion-Aware Modeling from Unsegmented LiDAR

This code extends the previous single-tree approach to a multi-tree setting. Instead of fitting one tree to an already segmented crown, it fits K tree crowns simultaneously to an unsegmented LiDAR point cloud. The method is occlusion-aware, meaning it models the fact that LiDAR returns depend on vegetation located above a point.

Input

The input is a full-scene LAZ/LAS point cloud without requiring tree segmentation.

Main inputs:

- `INPUT_LAZ`: unsegmented LiDAR file
- optional spatial subset (BBOX) to reduce computation
- optional classification field for ground filtering
- user-defined number of trees:
    - `K_TREES = 60`

Unlike Step 1, this code does not use `crown_id` or any segmentation labels.

Output

The output is a **GeoPackage** containing one record per estimated tree.
For each tree, the file stores:

- `tree_id`
- `x, y, z`: estimated tree center
- `radius_xy`: horizontal crown radius
- `radius_z`: vertical crown radius
- `lambda`: estimated foliage density
- `z_top = z + radius_z`

The geometry is a point at the estimated crown center in the XY plane.

For multiple trees, the total density is:

$$\Lambda(x, y, z) = \sum_i \lambda_i(x, y, z)$$

Each tree contributes density using:

- a constant density parameter $\lambda i | lambda_i$λi
- multiplied by a soft inside weight

This allows several crowns to overlap and jointly explain the observed LiDAR points.

Parameter Initialization

Because the point cloud is unsegmented, the code must initialize tree locations automatically.

1. Initial tree centers

The initialization is based on high canopy points:

- it selects points above the 85th height percentile
- then applies greedy farthest-point sampling in XY

This spreads the initial tree centers across the canopy and avoids placing many centers too close together.

2. Initial z-coordinate

The initial vertical center for all trees is set to the median scene height.

3. Initial radii

Initial values are set manually:

- `rxy0 = 3.0`
- `rz0` from the vertical spread of the scene
- `lam0 = 2.0`

These are rough starting values for optimization.

Optimization

The model is trained with PyTorch using the Adam optimizer.

Main settings:

- `EPOCHS = 1500`
- `LR = 0.03`

The objective is to maximize likelihood, which is equivalent to minimizing:

$$\mathrm{NLL} = -\frac{1}{N}\sum \log p(x_n)$$

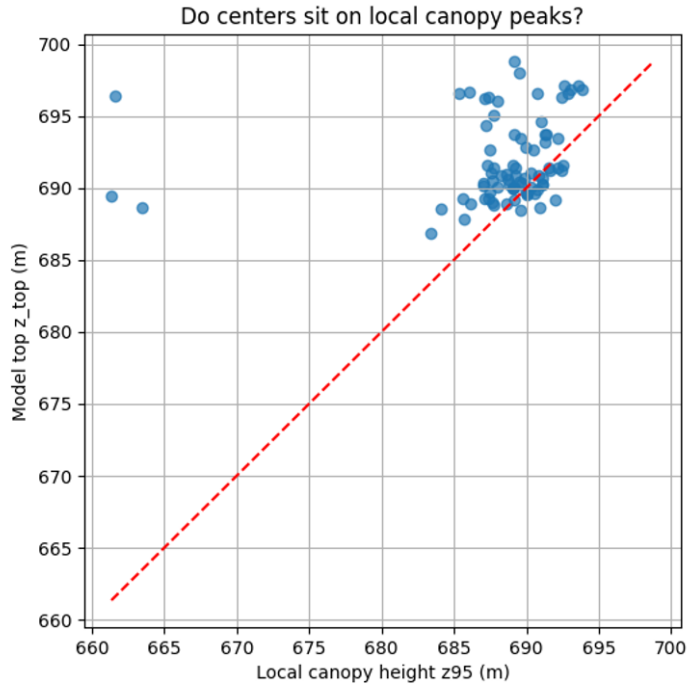
This method has several advantages:

- does not require pre-segmented crowns
- models multiple trees jointly
- accounts for canopy overlap
- uses a physically motivated attenuation model
- produces interpretable crown parameters

Some limitations are also important:

- the number of trees Kmust be chosen beforehand
- tree crowns are simplified as axis-aligned spheroids
- initialization may affect the final result
- optimization is computationally expensive
- results depend on the chosen bounding box and point sampling

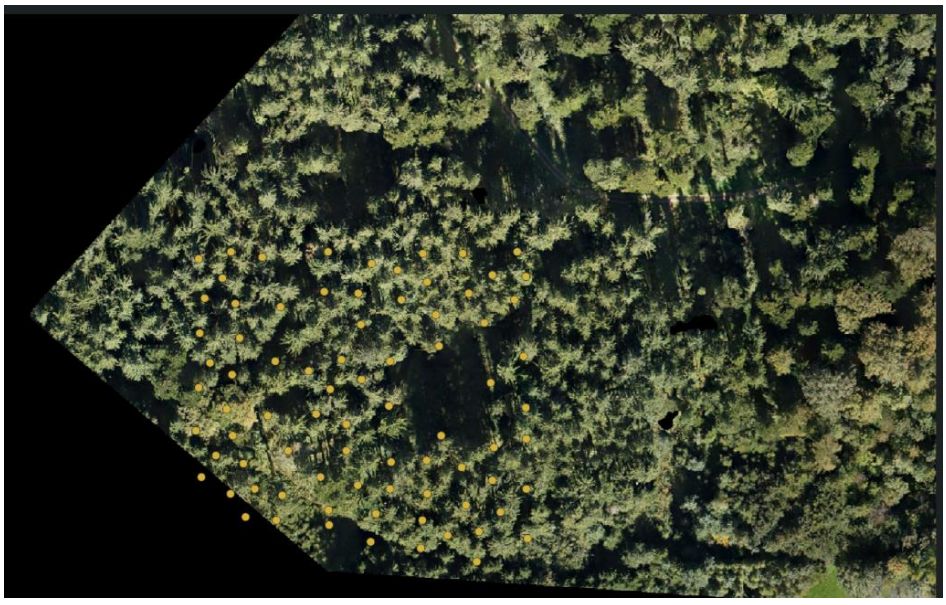
Evaluation of Tree Center Placement



Most points lie close to the 1:1 line, indicating that the fitted tree crowns generally align with local canopy maxima.

The majority of trees fall within approximately ±3–5 meters of the reference canopy height, suggesting that the model captures the main vertical structure of the canopy.

A few outliers appear where predicted heights deviate more strongly from the local canopy peak.



The estimated tree centers are generally located on individual canopy crowns, indicating that the model successfully identifies dominant tree positions within the forest stand. The spatial pattern of the points follows the visible canopy structure, with centers typically placed near the brightest and most elevated parts of the crowns.

In areas where the canopy is dense, some centers appear relatively close to each other. This may reflect overlapping crowns or closely spaced trees, which the model attempts to represent with multiple spheroidal crowns.

Some regions of the image contain fewer estimated centers, which correspond to canopy gaps, lower vegetation, or areas where LiDAR returns are sparse.

One clear weakness of the current model is that a single LiDAR point can be explained by several trees at the same time because the total density is computed as a sum of all tree contributions,

This creates ambiguous point ownership: neighboring crowns can both fit the same canopy returns, which can lead to duplicated tree centers or poor separation between adjacent trees.

Step 2.2: same as Step 2 but Initialized from Step 1 Results

The estimated tree centers are **densely clustered in the central region**, which corresponds to an area with a **high concentration of trees** visible in the aerial image. In contrast, fewer centers appear in the surrounding areas where the canopy is sparse or where clearings are present.

Step 2.3: softmax responsibilities

This code is an extension of the multi-tree Step 2 model for unsegmented LiDAR point clouds. It fits K tree crowns simultaneously while introducing softmax responsibilities, so that each LiDAR point is explained mainly by one tree instead of being shared equally by several nearby crowns.

The goal is to improve tree separation in dense canopy areas.

Input

The input is a **full-scene LAZ/LAS file** without any tree segmentation.

Main inputs are:

- `INPUT_LAZ`: unsegmented LiDAR point cloud
- optional `BBOX`: spatial subset for faster processing
- optional ground classification
- `K_TREES`: number of trees to fit

Unlike Step 1, no `crown_id` field is needed.

Output

The output is a **GeoPackage** containing one modeled tree per row, with:

- `tree_id`
- `x, y, z`: estimated center
- `radius_xy`: horizontal crown radius
- `radius_z`: vertical crown radius
- `lambda`: foliage density
- `z_top`: modeled crown top

The result is stored as point geometry at the estimated crown center.

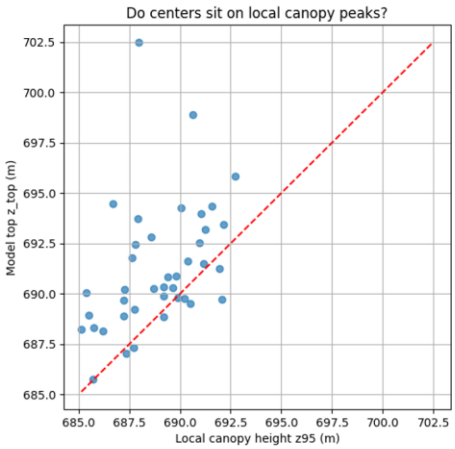
Important Difference from the Previous Step 2.1

The key difference is the introduction of **softmax responsibilities**.

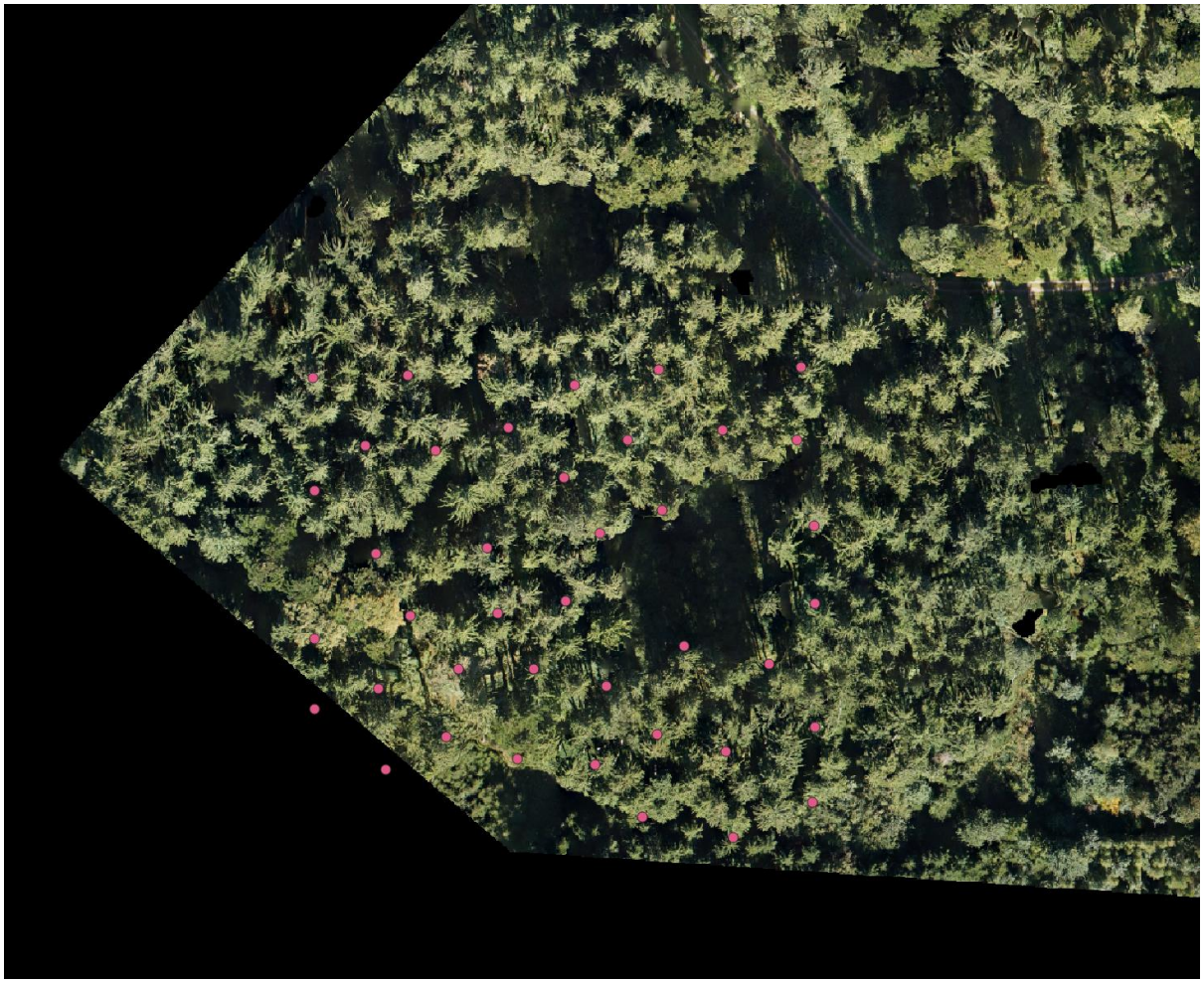
In the previous model, the total density was simply the sum of all tree contributions, so a point could be explained by several trees at once.
In this version, each point is explained mainly by the tree with the strongest response.

This improves:

- crown separation
- competition between neighboring trees
- reduction of duplicated tree centers



Most points lie close to the 1:1 line, indicating that the estimated tree centers generally align with local canopy peaks. Compared with the previous version without responsibilities, the points appear more consistently clustered around the reference line, suggesting better correspondence between modeled crowns and the actual canopy structure.



The estimated centers generally align with **individual canopy crowns**, indicating that the model successfully detects dominant trees in the stand. Compared with the previous version without responsibilities, the centers appear **more evenly distributed** across the canopy and less clustered within the same crown.